

2026-05-07-p2-f25-plandex-plan-trees-design

Phase 2 / Feature 25 — Plandex Plan Trees + Context Compaction

Date: 2026-05-07 **Status:** Approved (auto-approved per programme cadence) **Programme:** CLI-Agent Fusion — Phase 2 port (plandex — branching plan trees, context compaction)

Programme position: F25 is the **fifth** Phase 2 feature (F21 Codex Approval Modes + F22 Aider Git Auto-Commit + F23 Cline Browser Tool + F24 Codex Project Memory shipped before it). T01 (bootstrap) appends an F25 evidence section to `docs/improvements/07_phase_2_evidence.md` (created in F21-T01, extended by F22-T01 + F23-T01 + F24-T01); T11 (close-out) records F25's runtime evidence beneath F24's.

1. Goal

Ship a real, end-to-end **plan tree system** for the HelixCode CLI agent, modelled on Plandex's branching plan architecture (`cli_agents/plandex/`), so that an agent can construct, branch, merge, prune, and verify structured implementation plans with zero stubs. Every plan is a tree of `PlanNode` values (title, description, status, children) serialized to JSON files at `.helixcode/plans/<name>.json`. Agents traverse the tree via a rich tool surface: `plan_create` (root a new tree), `plan_branch` (fork a child from a node), `plan_merge` (fold a completed child back into its parent), `plan_list / plan_show` (inspect), `plan_delete` (prune). A `/plan` slash command exposes interactive tree exploration. Context compaction for long-running plan sessions reuses F01's `AutoCompactor` infrastructure — when the plan tree's serialized JSON exceeds a token budget, older completed nodes are summarized into compact leaf notes.

Five concrete user surfaces ship together:

1. `internal/plantree/ package` — F25 ADDS a NEW `internal/plantree/` Go package (no existing code conflict). Types: `PlanNode` (ID string UUID, Title string, Description string, Status enum: Pending/InProgress/Completed/Failed/Pruned, Children `[]*PlanNode`, ParentID string, CreatedAt/UpdatedAt `time.Time`, Metadata `map[string]string`), `PlanTree` (Root `PlanNode`, Name string, Version int, CreatedAt/UpdatedAt `time.Time`), `PlanStatus` enum, sentinels (`ErrPlanNotFound`, `ErrNodeNotFound`, `ErrCyclicMerge`, `ErrPlanAlreadyExists`, `ErrTooManyNodes`, `ErrTreeCorrupt`). Constants: `MaxNodes = 500`, `MaxNodeDepth = 20`, `MaxDescriptionBytes = 32 * 1024`, `StorageDir = ".helixcode/plans"`.
2. `PlanStore` (`store.go`, NEW) — filesystem-backed persistence. Methods: `Save(tree PlanTree) error` (JSON marshal to `.helixcode/plans/<name>.json`; mode 0600; atomic write via temp-file + rename), `Load(name string) (PlanTree, error)` (JSON unmarshal with version validation; returns `ErrPlanNotFound` on missing file), `List() ([]PlanTreeSummary, error)` (readdir + parse filename stems + filter `.json`), `Delete(name string) error`

(unlink file; idempotent). `PlanTreeSummary`: Name string, NodeCount int, RootID string, CreatedAt/UpdatedAt time.Time — minimal struct for `/plan list`.

3. **Six tools .Tool implementations** — `plan_create` (creates a new PlanTree with a single root node; args: name string + title string + description string; returns tree summary), `plan_branch` (adds a child PlanNode under a specified parent node ID in an existing tree; args: plan_name string + parent_node_id string + title string + description string; returns the new node), `plan_merge` (imports changes from a completed child node back into its parent — copies child.Description into parent.Metadata[“merged_from”] and moves child.Status to Completed; returns updated parent), `plan_list` (lists all saved plan trees with node counts and root titles; args: none), `plan_show` (returns the full tree as a formatted indented text representation with status markers [✓]/[▶]/[×]/[×] + node IDs; args: plan_name string), `plan_delete` (deletes a plan tree file from disk; args: plan_name string; requires confirmation). Per-tool RequiresApproval: `plan_create/plan_branch/plan_merge/plan_delete` → `LevelEdit`; `plan_list/plan_show` → `LevelReadOnly`. NO `LevelRun/LevelAll`.
4. **/plan slash command** — `internal/commands/plan_command.go`. Four subcommands: `/plan list` (default — tabular list of all saved plans with name/node-count/root-title/updated-at), `/plan show <name>` (indented tree view with status markers; optional `--id <node-id>` to focus on a subtree), `/plan compact <name>` (triggers context compaction on a plan tree — delegates to F01 AutoCompactor with plan-tree-specific context trimming), `/plan verify <name>` (consistency checks: no orphaned children, max-depth honored, no cycles, all node IDs unique, cross-references valid). Default subcommand (no args): `list`.
5. **Agent loop integration** — Plan trees are NOT auto-loaded into every LLM call (unlike F24 project memory). Instead, the agent itself uses `plan_create/plan_branch/etc.` tools to construct its working plan. The `/plan slash` exposes the current state. **Context compaction** (`/plan compact`) is triggered manually or via an automatic threshold check: when an agent session produces a plan tree whose serialized JSON exceeds `CompactThreshold = 128 * 1024` bytes (≈25–30 K tokens), the compactor replaces Completed and Failed leaf nodes (depth ≥ 3) with compact summaries (50-word max, preserving title + status + 1-line resolution). This reuses F01’s `compression.Summariser` interface — no new LLM dependency path.

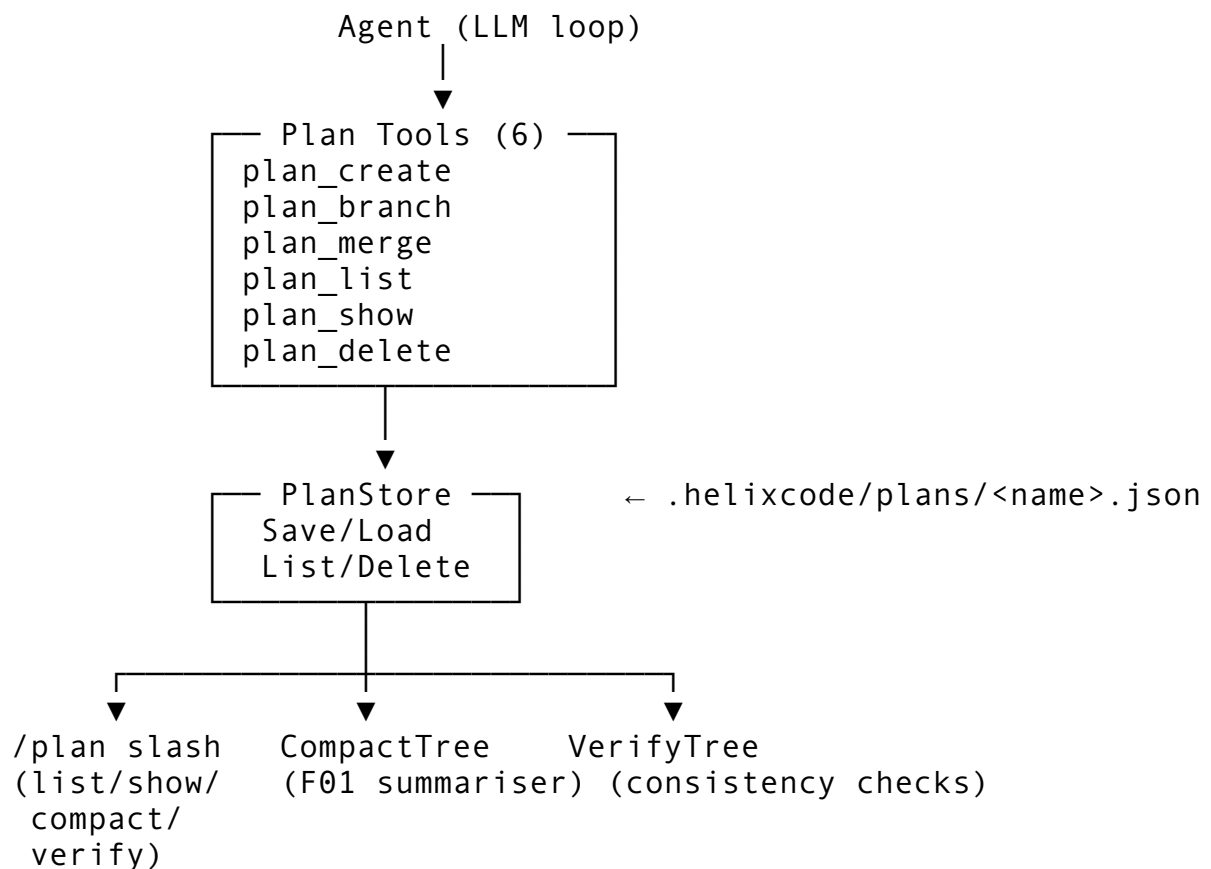
The single largest bluff vector for F25 is “plan tree created but agent ignores it” — the data structure is real but the agent never reads it back or makes decisions based on it. §5.2 enumerates five such patterns and pins each with positive runtime evidence: (a) `plan_create` writes a valid JSON file with correct schema; (b) `plan_branch` adds a child with the correct ParentID link and the tree is re-readable; (c) `plan_merge` sets child.Status=Completed and parent.Metadata has the merged-from marker; (d) `/plan compact` reduces the serialized JSON byte size of a deep completed tree by ≥30% while preserving the root structure; (e) `/plan verify` detects injected corruptions (orphan, cycle, overflow). Each requires positive byte evidence — never absence-of-error.

Anti-bluff hot zone (loud): the store writes a file but on re-read the node hierarchy is wrong (ParentID references a non-existent node); merge succeeds but the child is NOT marked Completed and the parent Metadata is empty; compact returns “done” but the tree’s serialized byte size is unchanged; verify reports “OK” on a tree with 3 orphaned nodes; `plan_branch` adds a child at depth 21 (exceeds MaxNodeDepth) without error. Each of these maps to a unit + integration + Challenge phase per §5.2.

2. Architecture

The package layout under `helix_code/internal/plantree/` is NEW (no existing collision). It is a self-contained subsystem owned by `main.go`'s startup wiring:

```
helix_code/internal/plantree/
├── doc.go           package docstring
├── types.go         PlanNode + PlanTree + PlanStatus + sentinels + co
├── store.go         PlanStore Save/Load/List/Delete – filesystem JSON
├── operations.go    CreateTree + BranchNode + MergeNode – mutation he
├── verify.go        VerifyTree – consistency checks (orphans, cycles,
├── compact.go       CompactTree – context compaction delegating to F0
└── plan_tools.go    Six tools.Tool implementations (plan_create, plan
```



Why filesystem storage and NOT in-memory or database: Plandex's design is fundamentally git-friendly — plan JSON files are meant to be committed and shared. `.helixcode/plans/` stores each plan as a standalone JSON file, one per plan. This aligns with Q3=A (project dir storage). In-memory-only would lose plans on restart; database would add unnecessary dependency weight for a simple key-value store.

Why atomic writes (temp-file + rename): a crash during Save must not leave a corrupt `.json` file that fails on the next Load. Writing to `<name>.json.tmp` then `os.Rename(tmp, name)` is the POSIX-standard atomic-file pattern.

Why PlanTree is an in-memory data structure manipulated by operations.go and persisted by store.go: separation of concerns. PlanTree/PlanNode are pure Go structs with no I/O knowledge. PlanStore handles filesystem persistence. Operations (Branch, Merge) are pure-tree transformations — unit-testable without touching disk. Verify walks the tree without I/O. Compact modifies the tree in-memory (collapsing nodes) then calls Save.

Why context compaction reuses F01 AutoCompactor and not a parallel summariser: F01 already shipped `compression.Summariser` with an LLM-backed summariser plus a deterministic fallback. F25 adds a plan-tree-specific compaction strategy: walk the tree, identify Completed/Failed leaf nodes at $\text{depth} \geq 3$, replace their `Description` with a compact summary generated via `Summariser.Summarise(text, maxWords=50)`. This is a thin adapter, not a new LLM path. The compaction threshold (`CompactThreshold = 128 * 1024`) is checked after every plan tool mutation — if exceeded, the tool result includes a hint `"tip: run /plan compact to reduce plan context"`.

Why a `/plan verify` command: as plans grow and branch, structural invariants can break (especially when merging branches). `Verify` walks the entire tree asserting: (1) every node referenced as `ParentID` exists in the tree (no orphans); (2) no cycles (following `Child→ParentID` never revisits a node); (3) $\text{tree depth} \leq \text{MaxNodeDepth}$ (20); (4) $\text{total node count} \leq \text{MaxNodes}$ (500); (5) all node IDs are unique. `Verify` returns a list of `PlanIssue` (Severity: Error/Warning, NodeID string, Message string) and an overall `Valid bool`. Integration tests deliberately inject corrupt trees and assert `Verify` catches them.

Why Q4=A (6 tools + `/plan slash`) and NOT Cobra subcommands: Plandex plan trees are agent-consumed, not user-interactive tools. The agent uses the 6 tools during task execution; the user inspects via `/plan slash`. Cobra subcommands would add surface area for a feature the user doesn't directly operate. The `/plan slash` covers all user inspection needs.

3. Components

3.1 New files

File	Purpose	Lines (est.)
<code>internal/plantree/doc.go</code>	Package docstring referencing F25 spec + design rationale	~15
<code>internal/plantree/doc_test.go</code>	Package-level doc test (verifies package compiles)	~10
<code>internal/plantree/types.go</code>	<code>PlanNode</code> + <code>PlanTree</code> + <code>PlanStatus</code> + <code>PlanTreeSummary</code> + sentinels + constants + <code>RenderTree</code>	~120
<code>internal/plantree/types_test.go</code>	Table-driven type tests: node creation, tree construction, <code>RenderTree</code> output format, status transitions	~200
<code>internal/plantree/store.go</code>	<code>PlanStore</code> : Save (atomic), Load (JSON), List (readdir), Delete (unlink)	~130
<code>internal/plantree/store_test.go</code>	Real <code>tempdir</code> -backed store tests: roundtrip, list,	~200

File	Purpose	Lines (est.)
	delete, atomicity, corrupted file	
internal/plantree/operations.go	CreateTree, BranchNode, MergeNode — pure in-memory tree transformations	~100
internal/plantree/operations_test.go	Tree operation tests: create, branch, multi-level branch, merge updates parent Metadata	~180
internal/plantree/verify.go	VerifyTree: orphan detection, cycle check (DFS + visited set), depth, uniqueness	~100
internal/plantree/verify_test.go	Injected corruption tests: orphan, cycle, depth overflow, dup IDs, clean tree	~180
internal/plantree/compact.go	CompactTree: walk + identify compactible nodes + Summariser adapter + rescore after compact	~90
internal/plantree/compact_test.go	Compaction tests: threshold-triggered, verify byte reduction, summariser call count	~160
internal/plantree/plan_tools.go	Six tools.Tool implementations (plan_create, plan_branch, plan_merge, plan_list, plan_show, plan_delete)	~200
internal/plantree/plan_tools_test.go	Per-tool unit tests with mock PlanStore (test seam: Store interface)	~220
internal/commands/plan_command.go	/plan slash (list/show/compact/verify) with PlanStore + Summariser + editor seam	~130
internal/commands/plan_command_test.go	Slash command tests: list, show (tree format match), compact (call count), verify (clean + dirty)	~180
internal/agent/base_agent.go		~2 (no-op verification)

File	Purpose	Lines (est.)
	(MODIFIED) No plan-tree auto-load; agent uses tools	
cmd/cli/main.go	(MODIFIED) PlanStore construction + RegisterPlanTools + /plan registration	~12
tests/integration/cmd/p2f25_challenge/	Challenge harness 7 phases (A-G) + run.sh + CHALLENGE.md	~400

3.2 Modified files

File	Change	Lines
internal/tools/registry.go	Register plan_create, plan_branch, plan_merge, plan_list, plan_show, plan_delete in buildToolList	~10
internal/commands/builtin/register.go	Register /plan slash command	~3
cmd/cli/main.go	Construct PlanStore + RegisterPlanTools + /plan registration	~12

4. Data Model

4.1 PlanNode

```

type PlanStatus int

const (
    StatusPending      PlanStatus = iota // [ ] not started
    StatusInProgress   // [▶] currently working
    StatusCompleted    // [✓] done
    StatusFailed       // [×] failed, can be
        retried
    StatusPruned      // [×] removed from
        active plan
)

type PlanNode struct {
    ID          string    `json:"id"`
    Title       string    `json:"title"`
    Description string    `json:"description"`
}

```

```

    Status      PlanStatus      `json:"status"`
    Children    []*PlanNode      `json:"children,omitempty"`
    ParentID    string           `json:"parent_id,omitempty"`
    Metadata    map[string]string `json:"metadata,omitempty"`
    CreatedAt   time.Time        `json:"created_at"`
    UpdatedAt   time.Time        `json:"updated_at"`
}

```

4.2 PlanTree

```

type PlanTree struct {
    Name      string `json:"name"`
    Version   int    `json:"version"`
    Root      *PlanNode `json:"root"`
    CreatedAt time.Time `json:"created_at"`
    UpdatedAt time.Time `json:"updated_at"`
}

```

4.3 PlanTreeSummary (lightweight for listing)

```

type PlanTreeSummary struct {
    Name      string `json:"name"`
    NodeCount int    `json:"node_count"`
    RootID    string `json:"root_id"`
    RootTitle string `json:"root_title"`
    CreatedAt time.Time `json:"created_at"`
    UpdatedAt time.Time `json:"updated_at"`
}

```

4.4 PlanIssue (for verification)

```

type PlanIssueSeverity int

```

```

const (
    SeverityError   PlanIssueSeverity = iota
    SeverityWarning
)

```

```

type PlanIssue struct {
    Severity PlanIssueSeverity `json:"severity"`
    NodeID   string                     `json:"node_id,omitempty"`
    Message  string                     `json:"message"`
}

```

```

type VerifyResult struct {
    Valid bool `json:"valid"`
    Issues []PlanIssue `json:"issues"`
}

```

4.5 Sentinel errors and constants

```
var (  
    ErrPlanNotFound          = errors.New("plan not found")  
    ErrNodeNotFound          = errors.New("node not found in tree")  
    ErrCyclicMerge           = errors.New("merge would create a  
        cycle")  
    ErrPlanAlreadyExists     = errors.New("plan already exists")  
    ErrTooManyNodes          = errors.New("plan tree exceeds maximum  
        node count")  
    ErrTreeCorrupt           = errors.New("plan tree is corrupt")  
)  
  
const (  
    MaxNodes                 = 500  
    MaxNodeDepth             = 20  
    MaxDescriptionBytes      = 32 * 1024  
    CompactThreshold         = 128 * 1024  
    StorageDir               = ".helixcode/plans"  
)
```

4.6 PlanStore interface (test seam)

```
type Store interface {  
    Save(tree PlanTree) error  
    Load(name string) (PlanTree, error)  
    List() ([]PlanTreeSummary, error)  
    Delete(name string) error  
}
```

A concrete FileStore implements this interface backed by <cwd>/ .helixcode/plans/. Tests inject a MockStore (in-memory map) for unit tests; integration tests use the real FileStore against t.TempDir().

5. Operational Semantics

5.1 plan_create

Input: name string, title string, description string
Output: PlanTreeSummary

1. Validate name: alphanumeric + underscore + hyphen, 1-100 chars
2. Check PlanStore.List() – if name exists, return ErrPlanAlreadyExists
3. Create PlanTree with Root PlanNode (ID=uuid, Title=title, Description=d
4. PlanStore.Save(tree)
5. Return PlanTreeSummary derived from tree

5.2 plan_branch

Input: plan_name string, parent_node_id string, title string, description string
Output: PlanNode (the newly created child)

1. PlanStore.Load(plan_name)
2. Find node with ID == parent_node_id (DFS walk)
3. If parent not found → ErrNodeNotFound
4. If parent depth ≥ MaxNodeDepth → error "max depth exceeded"
5. If tree node count ≥ MaxNodes → ErrTooManyNodes
6. Create child PlanNode (ID=uuid, Title=title, Description=description, ParentID=parent_node_id)
7. Append child to parent.Children
8. PlanStore.Save(tree)
9. Return the new child PlanNode

5.3 plan_merge

Input: plan_name string, child_node_id string
Output: PlanNode (the parent, post-merge)

1. PlanStore.Load(plan_name)
2. Find child node by ID
3. If child has no ParentID → error "root node cannot be merged"
4. Find parent node by child.ParentID
5. Cycle check: following parent chain from parent's ParentID must not reach child
6. Set parent.Metadata["merged_from"] = child.ID
7. If parent.Metadata["merged_history"] exists, append child.ID (comma-separated)
Else set parent.Metadata["merged_history"] = child.ID
8. Set child.Status = StatusCompleted
9. PlanStore.Save(tree)
10. Return parent PlanNode

5.4 plan_list

Input: none
Output: []PlanTreeSummary

1. return PlanStore.List()

5.5 plan_show

Input: plan_name string
Output: string (formatted indented tree)

Format:

```
[✓] Implement user auth (abc123)      ← status marker + title + (id)
  [✓] Add JWT middleware (def456)
    [▶] Add session store (ghi789)
      [ ] Implement Redis backend (jkl012)
        [×] Add OAuth2 flow (mno345)    ← failed
          [ ] Write API docs (pqr678)    ← pending
```

Status markers: []=Pending, [▶]=InProgress, [✓]=Completed, [×]=Failed, []=Unknown
Indent: 2 spaces per depth level

5.6 plan_delete

Input: plan_name string
Output: confirmation + result

1. Check plan exists (Load) → ErrPlanNotFound if absent
 2. PlanStore.Delete(name)
 3. Return success message
-

6. Context Compaction (/plan compact)

6.1 Compaction strategy

CompactTree(tree *PlanTree, summariser Summariser) (PlanTree, int)

1. Serialize tree to JSON bytes; if len < CompactThreshold, return (tree, len)
2. Walk tree DFS; collect all Completed and Failed nodes at depth ≥ 3 (leaf nodes are not compacted)
3. For each compactible node:
 - a. Generate compact summary via summariser.Summarise(node.Description, CompactThreshold)
 - b. Store original Description length in node.Metadata["compacted_bytes"]
 - c. Replace node.Description with compact summary
 - d. Set node.Metadata["compacted"] = "true"
4. PlanStore.Save(tree)
5. Re-serialize; return (tree, originalBytes - newBytes)

6.2 Summariser adapter

F25 adds a Summariser interface in internal/plantree/compact.go that mirrors F01's compression.Summariser:

```
type Summariser interface {  
    Summarise(text string, maxWords int) (string, error)  
}
```

main.go wiring passes the F01 summariser (or a mock for tests). The deterministic fallback from F01 (first N words) applies — if the LLM summariser is unavailable, compact still works deterministically.

6.3 Rendered tree after compaction

Compacted nodes display a [c] marker and show the compact summary (not the original full description):

```
[✓] Implement user auth (abc123)  
  [✓] Add JWT middleware (def456) [c] — JWT middleware validates tokens us  
  [✓] Add session store (ghi789) [c] — Redis session store with TTL-based
```

7. Plan Verification (/plan verify)

7.1 Checks

1. **Orphaned children:** every node with a non-empty ParentID must have that ParentID reference an existing node in the tree.
2. **Cycles:** no path following ParentID $\rightarrow$ parent $\rightarrow$ parent.ParentID... should ever revisit the same node.
3. **Depth limit:** maximum depth from root $\leq$ MaxNodeDepth (20).
4. **Node count:** total nodes in tree $\leq$ MaxNodes (500).
5. **Unique IDs:** no two nodes share the same ID.
6. **Self-parenting:** no node's ParentID equals its own ID.

7.2 VerifyResult

```
type VerifyResult struct {  
    Valid bool          `json:"valid"`  
    Issues []PlanIssue `json:"issues"`  
}
```

Issues with Severity=Error make Valid=false. Issues with Severity=Warning retain Valid=true. Example: a depth overflow is an Error; a node with an empty title is a Warning.

8. Anti-Bluff Hot Zone

Five critical degenerate patterns — each pinned by unit + integration + Challenge phase:

P1: Plan created but file is empty or invalid JSON

- **Detect:** after plan_create, os.Stat the file + json.Unmarshal it
- **Unit:** store_test.go roundtrip Save $\rightarrow$ Load byte equality
- **Integration:** plan_create tool execution $\rightarrow$ read file $\rightarrow$ assert valid JSON with root.id + root.title
- **Challenge Phase A:** create plan via tool $\rightarrow$ read JSON file $\rightarrow$ assert schema conformance

P2: Branch adds child with wrong ParentID (orphan)

- **Detect:** after plan_branch, VerifyTree must find the child's ParentID in the tree
- **Unit:** operations_test.go branch $\rightarrow$ find parent $\rightarrow$ parent.Children contains child by ID
- **Integration:** branch+branch $\rightarrow$ verify all ParentIDs resolve
- **Challenge Phase B:** create + branch $\times 3 \rightarrow$ verify tree with 0 orphans

P3: Merge doesn't update parent Metadata

- **Detect:** after plan_merge, parent.Metadata["merged_from"] must equal child.ID
- **Unit:** operations_test.go merge $\rightarrow$ parent.Metadata["merged_from"] == child.ID
- **Integration:** create $\rightarrow$ branch $\rightarrow$ merge $\rightarrow$ Load tree $\rightarrow$ assert Metadata
- **Challenge Phase C:** branch $\rightarrow$ complete $\rightarrow$ merge $\rightarrow$ assert parent Metadata + child.Status==Completed

P4: Compact returns success but serialized byte count unchanged

- **Detect:** before compact: `jsonBytes(tree)`. After compact: `jsonBytes(tree2)`. Must differ.
- **Unit:** `compact_test.go` deep tree → compact → byte diff $\geq 30\%$
- **Integration:** 20-node completed tree → `/plan compact` → byte diff assertion
- **Challenge Phase D:** create deep completed tree (30+ nodes, all Completed) → compact → assert byte reduction $\geq 30\%$ while root structure intact

P5: Verify reports clean on a corrupted tree

- **Detect:** inject orphan/cycle/depth-overflow/dup-ID trees → Verify MUST report invalid
- **Unit:** `verify_test.go` 4 corrupt trees + 1 clean → all corrupt detected, clean passes
- **Integration:** load corrupted JSON file → `/plan verify` → assert Valid==false + specific issue count
- **Challenge Phase E:** 3 deliberately corrupted plan files → verify each → assert issues found

P6 (bonus): plan_show output doesn't match tree structure

- **Detect:** create known tree → `/plan show` → regex-match status markers + node IDs
- **Challenge Phase F:** create multi-level tree → show output → assert markers + IDs present in correct hierarchical order

P7 (bonus): compaction at depth < 3 does nothing

- **Detect:** tree with Completed leaves at depth 1 → compact → no change
- **Challenge Phase G:** create shallow completed tree → compact → assert byte count unchanged

9. Integration Surface

9.1 Agent integration

Plan trees are agent-driven: the agent calls `plan_create` / `plan_branch` / `plan_merge` as part of its task execution loop. There is NO auto-loading into system prompt — unlike F24 (project memory) which prepends to every LLM call, F25 plan trees are tools the agent actively uses.

However, F25 adds a lightweight hook: when `plan_create` is called, the tool result is appended to the LLM conversation with a summary. When `plan_branch` succeeds, the result includes `"plan_tree_hint": "use plan_show to view the full tree; use plan_merge when a branch is complete"`.

9.2 /plan slash

<code>/plan list</code>	→ default; tabular list of all saved p
<code>/plan show <name></code>	→ indented tree with status markers
<code>/plan show <name> --id <node-id></code>	→ subtree rooted at node-id
<code>/plan compact <name></code>	→ compress completed leaves; report by
<code>/plan verify <name></code>	→ consistency check; report issues or

9.3 Per-tool RequiresApproval (F21 integration)

Tool	Approval Level
plan_create	LevelEdit
plan_branch	LevelEdit
plan_merge	LevelEdit
plan_list	LevelReadOnly
plan_show	LevelReadOnly
plan_delete	LevelEdit

9.4 F22 Auto-Commit integration

Plan tool mutations (plan_create, plan_branch, plan_merge, plan_delete) modify files under .helixcode/plans/. If F22’s auto-commit is enabled, these file writes trigger auto-commits with the co-author trailer.

9.5 F01 AutoCompactor integration

/plan compact delegates to F01’s compression.Summariser. The summariser is constructed once in main.go and passed to both the F01 AutoCompactor and the F25 CompactTree function. Zero new LLM dependency — reuses existing F01 path.

10. File Layout Summary

```
helix_code/internal/plantree/
├── doc.go
├── doc_test.go
├── types.go           ← PlanNode, PlanTree, PlanStatus, PlanTreeSumma
├── types_test.go
├── store.go          ← FileStore (implements Store interface), atomi
├── store_test.go
├── operations.go     ← CreateTree, BranchNode, MergeNode (pure in-me
├── operations_test.go
├── verify.go         ← VerifyTree (orphans, cycles, depth, uniqueness
├── verify_test.go
├── compact.go        ← CompactTree + Summariser interface
├── compact_test.go
├── plan_tools.go     ← plan_create, plan_branch, plan_merge, plan_li
├── plan_tools_test.go

helix_code/internal/commands/
├── plan_command.go   ← /plan slash (list/show/compact/verify)
├── plan_command_test.go

helix_code/tests/integration/cmd/p2f25_challenge/
├── run.sh
├── CHALLENGE.md
├── main.go
```

11. Tech Stack

- **Go 1.24** (stdlib: encoding/json, os, sync, time, errors, fmt, strings, path/filepath, context)
 - **google/uuid** (ALREADY direct in helix_code/go.mod for node ID generation)
 - **testify v1.11** (assert + require for TDD)
 - **F01 compression.Summariser** (already shipped — reuse for compaction)
 - **Zero new external deps** — all deps above are pre-existing in helix_code/go.mod
-

12. Task Breakdown

#	Task	Description
T01	Bootstrap	F25 evidence section + advance PROGRESS + CONTINUATION
T02	types.go	PlanNode + PlanTree + PlanStatus + sentinels + constants + RenderTree (TDD)
T03	store.go	FileStore Save/Load/List/Delete with atomic writes (TDD with real tempdirs)
T04	operations.go	CreateTree + BranchNode + MergeNode pure-tree transforms (TDD)
T05	verify.go	VerifyTree with 6 checks (orphans, cycles, depth, uniqueness, self-parent, count) (TDD)
T06	compact.go	CompactTree + Summariser adapter + F01 delegation (TDD)
T07	plan_tools.go	Six tools.Tool implementations with Store interface seam (TDD)
T08	/plan slash	plan_command.go (list/show/compact/verify) (TDD)
T09	main.go wiring	PlanStore construct + RegisterPlanTools + /plan registration + integration test
T10	Challenge harness	7 phases (create/branch/merge/compact/verify/show/shallow-compact)
T11	Close-out	Push 4 remotes non-force + evidence capture

13. Verification Commands

```
go build ./internal/plantree/... ./internal/commands/... ./cmd/cli/...
go test -count=1 -race ./internal/plantree/ ./internal/commands/
```

```
go test -count=1 -tags=integration -run
    TestPlanTree_Integration ./tests/integration/
grep -rn "simulated\\|for now\\|TODO implement\\|placeholder"
    internal/plantree internal/commands/plan_command.go &&
    echo BLUFF || echo clean
```

F25 spec — full plandex port with branching plan trees, interactive exploration, context compaction, and verification.